

Git/GitHub Complete Guide for Software Testers

Table of Contents

1. [Why Git/GitHub Matters for Testers](#)
2. [Git Fundamentals](#)
3. [Initial Setup](#)
4. [Essential Git Commands](#)
5. [GitHub Workflow for Testers](#)
6. [Testing-Specific Workflows](#)
7. [Collaboration Best Practices](#)
8. [Advanced Features for Testing Teams](#)
9. [Common Scenarios & Solutions](#)
10. [Best Practices & Tips](#)

Why Git/GitHub Matters for Testers

Key Benefits for QA Professionals

- **Version Control:** Track changes in test cases, automation scripts, and documentation
- **Collaboration:** Work seamlessly with developers and other testers
- **Bug Tracking:** Link test results to specific code versions
- **Test Management:** Organize test suites, data, and reports
- **CI/CD Integration:** Trigger automated tests on code changes
- **Documentation:** Maintain test plans, procedures, and knowledge base

What Testers Track in Git

Testing Repository Structure

```
├── test-cases/  
│   ├── functional/  
│   ├── regression/  
│   └── api/  
├── automation-scripts/  
│   ├── selenium/  
│   ├── api-tests/  
│   └── performance/
```

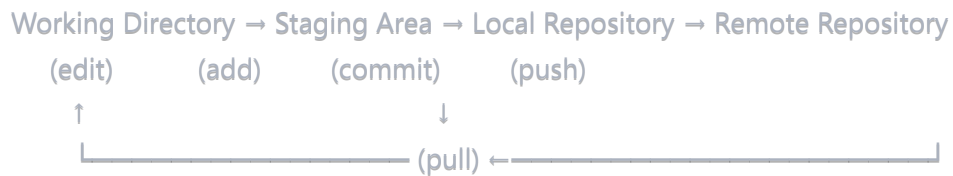
```
|— test-data/  
|— documentation/  
|— reports/  
|— tools-config/
```

Git Fundamentals

Core Concepts

- **Repository (Repo):** Project folder containing all files and history
- **Commit:** Snapshot of changes with a descriptive message
- **Branch:** Parallel version of code for different features/testing
- **Merge:** Combining changes from different branches
- **Pull Request:** Proposal to merge changes (code review process)
- **Clone:** Copy a repository to your local machine
- **Fork:** Personal copy of someone else's repository

Git Workflow Visualization



Initial Setup

1. Install Git

Windows:

- Download from git-scm.com
- Install with default settings

Mac:

```
bash  
  
# Using Homebrew  
brew install git  
  
# Or download from git-scm.com
```

Linux:

```
bash

# Ubuntu/Debian
sudo apt update
sudo apt install git

# CentOS/RHEL
sudo yum install git
```

2. Configure Git (First Time Setup)

```
bash

# Set your identity
git config --global user.name "Your Name"
git config --global user.email "your.email@company.com"

# Set default branch name
git config --global init.defaultBranch main

# Set default editor (optional)
git config --global core.editor "code --wait" # For VS Code

# Verify configuration
git config --list
```

3. Create GitHub Account

1. Go to github.com
2. Sign up with your professional email
3. Choose a professional username
4. Set up two-factor authentication (recommended)

4. Set Up SSH Keys (Recommended)

```
bash
```

```
# Generate SSH key
ssh-keygen -t ed25519 -C "your.email@company.com"

# Start SSH agent
eval "$(ssh-agent -s)"

# Add SSH key to agent
ssh-add ~/.ssh/id_ed25519

# Copy public key to clipboard (Mac)
pbcopy < ~/.ssh/id_ed25519.pub

# Copy public key to clipboard (Windows)
clip < ~/.ssh/id_ed25519.pub

# Copy public key to clipboard (Linux)
xclip -selection clipboard < ~/.ssh/id_ed25519.pub
```

Then add the key to GitHub:

1. Go to GitHub Settings → SSH and GPG keys
2. Click "New SSH Key"
3. Paste your public key
4. Test connection: `ssh -T git@github.com`

Essential Git Commands

1. Repository Operations

```
bash
```

Initialize new repository

`git init`

Clone existing repository

`git clone https://github.com/username/repo-name.git`

Clone to specific folder

`git clone https://github.com/username/repo-name.git my-project`

Check repository status

`git status`

View repository information

`git remote -v`

`git branch -a`

2. Basic File Operations

bash

Add files to staging area

`git add filename.txt` *# Add specific file*

`git add .` *# Add all files*

`git add *.js` *# Add all JavaScript files*

`git add test-cases/` *# Add entire directory*

Remove files

`git rm filename.txt` *# Remove file*

`git rm --cached filename.txt` *# Remove from tracking, keep locally*

View changes

`git diff` *# See unstaged changes*

`git diff --staged` *# See staged changes*

`git diff HEAD~1` *# Compare with previous commit*

3. Committing Changes

bash

Commit with message

```
git commit -m "Add login test cases for user authentication"
```

Commit all tracked files

```
git commit -am "Update regression test suite"
```

Amend last commit (fix message or add files)

```
git commit --amend -m "Corrected: Add login test cases"
```

View commit history

```
git log # Full history
```

```
git log --oneline # Condensed history
```

```
git log --graph --oneline # Visual branch history
```

4. Branch Management

bash

List branches

```
git branch # Local branches
```

```
git branch -a # All branches (local + remote)
```

Create new branch

```
git branch feature/api-testing
```

```
git checkout -b feature/ui-automation # Create and switch
```

Switch branches

```
git checkout main
```

```
git switch feature/api-testing # Modern alternative
```

Merge branch

```
git checkout main
```

```
git merge feature/api-testing
```

Delete branch

```
git branch -d feature/api-testing # Safe delete
```

```
git branch -D feature/api-testing # Force delete
```

5. Remote Operations

bash

Add remote repository

`git remote add origin https://github.com/username/repo.git`

Push changes

`git push origin main`

Push to main branch

`git push -u origin feature/testing`

Push and set upstream

Pull changes

`git pull origin main`

Pull from main branch

`git pull`

Pull from tracked branch

Fetch updates (without merging)

`git fetch origin`

GitHub Workflow for Testers

1. Setting Up a Testing Repository

bash

```
# Create new testing repository
mkdir qa-automation-project
cd qa-automation-project
git init
git remote add origin https://github.com/your-username/qa-automation-project.git

# Create initial structure
mkdir -p {test-cases/{functional,regression,api},automation-scripts,test-data,reports,documentation}
touch README.md .gitignore

# Create .gitignore for testing projects
echo "# Test Reports
reports/
logs/
screenshots/
*.log

# IDE files
.vscode/
.idea/

# OS files
.DS_Store
Thumbs.db

# Dependencies
node_modules/
__pycache__/
*.pyc" > .gitignore
```

2. Creating Your First Test Case Repository

Step 1: Create README.md

```
markdown
```


QA Automation Project

Overview

This repository contains automated test cases and manual testing documentation for [Project Name].

Structure

- ``test-cases/`` - Manual test cases organized by feature
- ``automation-scripts/`` - Automated test scripts
- ``test-data/`` - Test data files and fixtures
- ``reports/`` - Test execution reports
- ``documentation/`` - Testing procedures and guidelines

Getting Started

1. Clone repository: ``git clone [repo-url]``
2. Install dependencies: [installation steps]
3. Run tests: [execution commands]

Contributing

Please follow our testing standards and submit pull requests for review.

Step 2: Commit Initial Setup

bash

```
git add .  
git commit -m "Initial repository setup with testing structure"  
git push -u origin main
```

3. Feature Branch Workflow for Testing

bash

Start new testing feature

`git checkout main`

`git pull origin main`

`git checkout -b test/user-registration-validation`

Work on test cases

Create/edit files in your branch

Stage and commit changes

`git add test-cases/functional/user-registration.md`

`git commit -m "Add comprehensive user registration test cases"`

- Email validation scenarios

- Password strength requirements

- Field boundary testing

- Error message validation"

Push branch to GitHub

`git push -u origin test/user-registration-validation`

4. Creating Pull Requests

1. Push your branch to GitHub
2. Go to GitHub repository
3. Click "Compare & pull request"
4. Fill out PR template:

markdown

Test Case Updates

What's Changed

- Added user registration validation test cases
- Covered email format validation
- Added boundary testing scenarios

Testing Done

- Reviewed test cases with BA team
- Validated against requirements v2.1
- Cross-checked with existing regression suite

Checklist

- [x] Test cases follow company standards
- [x] All scenarios covered
- [x] Documentation updated
- [x] Ready for review

Testing-Specific Workflows

1. Bug Report Workflow

bash

Create bug report branch

`git checkout -b bug/login-timeout-issue`

Document bug

`mkdir -p bugs/2024-03`

`cat > bugs/2024-03/login-timeout-bug.md << EOF`

Bug Report: Login Timeout Issue

****Bug ID**:** BUG-2024-001

****Date**:** \$(date)

****Reporter**:** Your Name

Description

User login times out after 30 seconds on slow connections

Steps to Reproduce

1. Navigate to login page
2. Enter valid credentials
3. Click login on slow network (< 1Mbps)

Expected Result

Login completes within reasonable time

Actual Result

Login times out after 30 seconds

Environment

- Browser: Chrome 120
- OS: Windows 11
- Network: Slow 3G simulation

Screenshots

See: screenshots/login-timeout-2024-03-15.png

EOF

Commit bug report

`git add bugs/2024-03/login-timeout-bug.md`

`git commit -m "Bug Report: Login timeout on slow connections"`

- Documented steps to reproduce
- Added environment details
- Included screenshot evidence
- **Severity: Medium, Priority: High**

```
git push -u origin bug/login-timeout-issue
```

2. Test Data Management

```
bash
```

```
# Create test data branch
```

```
git checkout -b data/user-accounts-update
```

```
# Add test data
```

```
mkdir -p test-data/users
```

```
cat > test-data/users/valid-users.json << EOF
```

```
{  
  "valid_users": [  
    {  
      "username": "testuser1",  
      "email": "testuser1@example.com",  
      "password": "ValidPass123!",  
      "role": "standard"  
    },  
    {  
      "username": "admin_user",  
      "email": "admin@example.com",  
      "password": "AdminPass456!",  
      "role": "admin"  
    }  
  ]  
}
```

```
EOF
```

```
# Commit test data
```

```
git add test-data/users/valid-users.json
```

```
git commit -m "Add standardized test user data"
```

- Created valid user accounts for testing
- Includes standard and admin roles
- Follows password policy requirements"

```
git push -u origin data/user-accounts-update
```

3. Test Automation Script Management

bash

Create automation branch

```
git checkout -b automation/login-tests
```

Add automation script

```
mkdir -p automation-scripts/selenium
```

```
cat > automation-scripts/selenium/login_test.py << EOF
```

```
"""
```

Login Test Automation Script

Author: Your Name

Date: \$(date)

```
"""
```

```
import pytest
```

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
class TestLogin:
```

```
    def setup_method(self):
```

```
        self.driver = webdriver.Chrome()
```

```
        self.driver.get("https://app.example.com/login")
```

```
    def test_valid_login(self):
```

```
        """Test successful login with valid credentials"""
```

```
        # Test implementation here
```

```
        pass
```

```
    def test_invalid_login(self):
```

```
        """Test login failure with invalid credentials"""
```

```
        # Test implementation here
```

```
        pass
```

```
    def teardown_method(self):
```

```
        self.driver.quit()
```

```
EOF
```

Commit automation script

```
git add automation-scripts/selenium/login_test.py
```

```
git commit -m "Add Selenium login test automation"
```

- Created base login test class

- Added valid/invalid login scenarios

- Included proper setup/teardown methods"

```
git push -u origin automation/login-tests
```

Collaboration Best Practices

1. Working with Development Team

```
bash

# Stay updated with main branch
git checkout main
git pull origin main

# Create feature branch from latest main
git checkout -b test/new-feature-validation

# Regularly sync with main to avoid conflicts
git checkout main
git pull origin main
git checkout test/new-feature-validation
git merge main
```

2. Code Review Process for Testers

Reviewing Test Cases/Scripts:

```
bash

# Fetch latest changes
git fetch origin

# Check out PR branch for local review
git checkout -b review/pr-123 origin/feature/api-testing

# Review files
git diff main...feature/api-testing

# Add review comments locally
git add -A
git commit -m "Review feedback: Add negative test scenarios"
git push origin review/pr-123
```


3. Handling Merge Conflicts

```
bash

# When merge conflict occurs
git merge main
# Auto-merging test-cases/login.md
# CONFLICT (content): Merge conflict in test-cases/login.md

# Open conflicted file and resolve
# Look for conflict markers:
# <<<<<< HEAD
# Your changes
# =====
# Incoming changes
# >>>>>> main

# After resolving conflicts
git add test-cases/login.md
git commit -m "Resolve merge conflict in login test cases"
git push origin feature/login-testing
```

Advanced Features for Testing Teams

1. GitHub Issues for Test Management

Create Test Case Template:

```
markdown
```

```
---
name: Test Case Template
about: Template for documenting test cases
title: '[TEST] '
labels: test-case
assignees: ''
---
```

Test Case Information

```
**Test ID**: TC-XXX
**Priority**: High/Medium/Low
**Type**: Functional/Regression/Smoke
```

Test Scenario

Brief description of what is being tested

Preconditions

- System requirements
- Test data requirements
- User permissions needed

Test Steps

1. Step one
2. Step two
3. Step three

Expected Results

- What should happen after each step
- Final expected outcome

Test Data

- Input data required
- Expected outputs

Environment

- Browser/Device requirements
- Operating system
- Network conditions

2. GitHub Actions for Test Automation

Create `.github/workflows/test-automation.yml`:

yaml

name: Automated Testing

on:

push:

branches: [main, develop]

pull_request:

branches: [main]

jobs:

test:

runs-on: ubuntu-latest

steps:

- **uses:** actions/checkout@v3

- **name:** Set up Python

uses: actions/setup-python@v3

with:

python-version: '3.9'

- **name:** Install dependencies

run: |

pip install -r requirements.txt

- **name:** Run tests

run: |

pytest automation-scripts/ --html=reports/report.html

- **name:** Upload test results

uses: actions/upload-artifact@v3

if: always()

with:

name: test-results

path: reports/

3. Project Boards for Test Planning

Setting up Test Project Board:

1. Go to repository → Projects → New Project
2. Choose "Automated kanban" template

3. Create columns:

- **Backlog:** Test cases to be created
- **In Progress:** Currently writing/executing
- **Review:** Ready for peer review
- **Done:** Completed and approved

Common Scenarios & Solutions

1. Accidentally Committed Sensitive Data

```
bash

# Remove file from last commit
git rm --cached sensitive-data.txt
git commit --amend -m "Remove sensitive test data"

# Remove from entire history (use carefully!)
git filter-branch --force --index-filter \
'git rm --cached --ignore-unmatch sensitive-data.txt' \
--prune-empty --tag-name-filter cat -- --all
```

2. Need to Undo Last Commit

```
bash

# Undo commit but keep changes
git reset --soft HEAD~1

# Undo commit and discard changes
git reset --hard HEAD~1

# Undo pushed commit (creates new commit)
git revert HEAD
git push origin main
```

3. Working with Large Test Files

```
bash
```

```
# Use Git LFS for large files
git lfs install
git lfs track "*.mp4"
git lfs track "test-data/large-dataset.csv"
git add .gitattributes
git add test-videos/demo.mp4
git commit -m "Add large test video using Git LFS"
```

4. Cherry-picking Test Fixes

```
bash

# Apply specific commit to another branch
git checkout main
git cherry-pick abc1234 # commit hash from feature branch

# Cherry-pick without committing (review first)
git cherry-pick --no-commit abc1234
git status # Review changes
git commit -m "Apply critical test fix from feature branch"
```

Best Practices & Tips

1. Commit Message Standards for Testers

Good commit messages:

```
bash
```

Feature commits

"Add API authentication test suite"

"Update user registration validation scenarios"

Bug fix commits

"Fix: Resolve timeout in payment gateway tests"

"Bug fix: Handle null response in API test"

Documentation commits

"Docs: Update test execution procedures"

"Update: Refresh environment setup guide"

Data commits

"Data: Add new test users for performance testing"

"Update: Refresh test database schemas"

Commit message template:

Type: Brief description (50 chars max)

Detailed explanation of what and why (72 chars per line):

- What was changed
- Why it was necessary
- Any side effects or considerations

Closes #123 (if fixing an issue)

2. Branch Naming Conventions

bash

Feature testing

feature/user-authentication-tests

feature/payment-gateway-validation

Bug investigation

bug/login-timeout-investigation

bug/cart-calculation-error

Test data updates

data/user-accounts-refresh

data/product-catalog-update

Documentation

docs/test-procedures-update

docs/environment-setup-guide

Automation scripts

automation/selenium-login-suite

automation/api-performance-tests

3. Repository Organization

```
qa-project/  
├── .github/  
│   ├── workflows/      # CI/CD automation  
│   ├── ISSUE_TEMPLATE/ # Issue templates  
│   └── pull_request_template.md  
├── automation-scripts/  
│   ├── api/  
│   ├── ui/  
│   ├── performance/  
│   └── requirements.txt  
├── test-cases/  
│   ├── functional/  
│   ├── regression/  
│   ├── smoke/  
│   └── exploratory/  
├── test-data/  
│   ├── users/  
│   ├── products/  
│   └── configurations/  
└── documentation/
```

```
| |— test-plans/
| |— procedures/
| |— guidelines/
|— tools/
| |— scripts/
|— reports/      # Git ignored
|— .gitignore
|— README.md
```

4. Security Best Practices

```
bash

# Never commit sensitive data
echo "# Sensitive Data" >> config/secrets.json
echo ".env" >> .gitignore
echo "*.key" >> .gitignore
echo "passwords.txt" >> .gitignore
echo "test-data/real-user-data.csv" >> .gitignore

# Use environment variables for secrets
export DB_PASSWORD="your-password"
export API_KEY="your-api-key"

# Audit repository for secrets
git log --all --full-history -- "*.key"
```

5. Performance Tips

```
bash

# Speed up clones with shallow history
git clone --depth 1 https://github.com/user/repo.git

# Use sparse checkout for large repos
git config core.sparseCheckout true
echo "test-cases/" > .git/info/sparse-checkout
git read-tree -m -u HEAD

# Clean up repository
git gc --aggressive --prune=now
```


Quick Reference Card

Daily Commands

bash

```
git status          # Check current state
git add .           # Stage all changes
git commit -m "message"  # Commit with message
git push            # Push to remote
git pull            # Get latest changes
```

Branch Operations

bash

```
git branch          # List branches
git checkout -b new-branch  # Create and switch branch
git merge branch-name  # Merge branch
git branch -d branch-name  # Delete branch
```

Troubleshooting

bash

```
git log --oneline    # View commit history
git diff             # See changes
git reset --soft HEAD~1  # Undo last commit
git stash            # Temporarily save changes
git stash pop        # Restore stashed changes
```

Collaboration

bash

```
git fetch origin     # Get remote updates
git remote -v        # View remotes
git blame filename    # See who changed what
git show commit-hash  # View specific commit
```

Remember: Git is a powerful tool that becomes second nature with practice. Start with basic operations and gradually incorporate advanced features as your confidence grows. The key is consistent daily use and not being afraid to experiment in test repositories!